FACULDADE DE TECNOLOGIA PROFESSOR JESSEN VIDAL
FATEC SÃO JOSÉ DOS CAMPOS
CURSO SUPERIOR DE TECNOLOGIA EM DESENVOLVIMENTO DE SOFTWARE MULTIPLATAFORMA

Eduardo Fontes
Eduardo Kuwahara
Fábio Hiroshi
João Rossi
Kathellyn Caroline
Paulo Henrique
Victor Daniel

TESTES NÃO FUNCIONAIS: TESTES DE SEGURANÇA (SECURITY TESTING)

Roteiro detalhado da pesquisa semestral e contexto do exemplo prático aplicado a uma API REST de gestão de notas acadêmicas

Roteiro apresentado à disciplina de Qualidade e Testes de Software, do 6º semestre do curso de Desenvolvimento de Software Multiplataforma, ministrada pelo Prof. Lucas Nadalete, como entrega parcial da pesquisa semestral.

São José dos Campos
2026

SUMÁRIO

1 INTRODUÇÃO 3

2 ROTEIRO DETALHADO DOS TÓPICOS DA PESQUISA 3

2.1 Introdução sobre o tema 3

2.2 Histórico 4

2.3 O que é? 4

2.4 Qual o propósito? 5

2.5 Principais vantagens e desvantagens 5

2.6 Exemplos de ferramentas e frameworks 5

2.7 Ilustração do exemplo prático 6

2.8 Demonstração do exemplo prático 6

2.9 Referências 6

3 CONTEXTO DESCRITIVO DO PROBLEMA E DA SOLUÇÃO 7

3.1 Cenário: a NotasAPI 7

3.2 O problema 7

3.3 A solução proposta 8

3.4 Casos de teste planejados 9

3.5 Resultados esperados 10

4 CRONOGRAMA ATÉ A APRESENTAÇÃO 11

REFERÊNCIAS 12

1 INTRODUÇÃO

Este documento apresenta o roteiro detalhado da pesquisa semestral da equipe sobre o tema **Testes Não Funcionais: Testes de Segurança** (*Security Testing*), bem como o contexto descritivo do problema e da solução sobre os quais o exemplo prático será implementado. O objetivo é fixar, desde já, o escopo da pesquisa teórica, a sequência da apresentação de 26/11/2026 e o recorte técnico da demonstração.

Como objeto do exemplo prático, a equipe desenvolverá a **NotasAPI**, uma API REST simples de gestão de notas acadêmicas, construída especificamente para esta pesquisa em Python com o framework FastAPI. Optou-se por uma aplicação própria, e não por um sistema já em produção, por três motivos: (i) permite introduzir vulnerabilidades de forma proposital e controlada, sem colocar em risco dados ou serviços reais; (ii) o escopo reduzido deixa a demonstração clara e reproduzível em sala; e (iii) o domínio escolhido (notas de alunos, com papéis de aluno e professor) reúne as falhas mais comuns em APIs, como autenticação fraca, acesso a dados de outros usuários e injeção de SQL. A linguagem Python foi escolhida por concentrar ferramentas maduras e gratuitas de teste de segurança (Bandit, Semgrep, pip-audit) e pela geração automática da especificação OpenAPI pelo FastAPI, que alimenta diretamente o OWASP ZAP.

O documento está organizado da seguinte forma: a Seção 2 traz o roteiro de cada tópico exigido para a pesquisa e a apresentação; a Seção 3 descreve o problema, a solução proposta, a ilustração do exemplo e os casos de teste planejados; a Seção 4 apresenta o cronograma até a apresentação; e, por fim, são listadas as referências que servirão de base para a pesquisa.

2 ROTEIRO DETALHADO DOS TÓPICOS DA PESQUISA

Cada subseção a seguir corresponde a um dos pontos obrigatórios da pesquisa e da apresentação, com os subtópicos que serão abordados e a abordagem prevista.

2.1 Introdução sobre o tema

- Contextualização: crescimento de ataques a aplicações web e APIs e o custo de corrigir falhas de segurança tardiamente no ciclo de vida do software.
- Posicionamento do tema na classificação de testes: testes funcionais (o que o sistema faz) versus testes não funcionais (como o sistema se comporta: desempenho, usabilidade, segurança etc.).

- Segurança como característica de qualidade de produto na ISO/IEC 25010 (confidencialidade, integridade, não repúdio, responsabilização, autenticidade e resistência).
- Apresentação breve do sistema que será usado como exemplo (NotasAPI) e do que será demonstrado.

2.2 Histórico

O histórico será apresentado em formato de linha do tempo, destacando os marcos que levaram da invasão controlada feita por especialistas até a automação dos testes de segurança nas esteiras de integração contínua:

Período	**Marco a ser abordado**
Década de 1970	Relatório de Anderson (1972) para a Força Aérea dos EUA e uso de *tiger teams*: origem do teste de invasão (*penetration testing*) como prática formal.
1988	Worm de Morris (SPAFFORD, 1989): primeiro grande incidente na Internet, evidenciando a necessidade de testar vulnerabilidades como estouro de buffer.
Década de 1990	Surgimento de scanners automatizados de vulnerabilidades e popularização da Web, que amplia a superfície de ataque das aplicações.
2001-2004	Criação da comunidade OWASP, primeira lista OWASP Top 10 e artigo de Potter e McGraw (2004) propondo testes de segurança baseados em risco, integrados ao processo de testes.
2006	Publicação do *Security Development Lifecycle* da Microsoft (HOWARD; LIPNER, 2006): segurança incorporada a todas as fases do desenvolvimento.
2008-2010s	Guia NIST SP 800-115, consolidação de SAST/DAST/SCA e movimento DevSecOps (*shift-left*).
2019-atual	OWASP API Security Top 10 e OWASP Top 10 para aplicações com LLM, refletindo arquiteturas baseadas em APIs e o uso de IA generativa.

2.3 O que é?

- Definição: conjunto de técnicas que verificam se o software protege dados e funcionalidades conforme o esperado e se resiste a usos maliciosos, procurando vulnerabilidades antes que sejam exploradas (FELDERER et al., 2016; POTTER; MCGRAW, 2004).
- Diferença entre teste funcional de segurança (o controle existe e funciona, ex.: login exige senha) e teste de vulnerabilidade baseado em risco (o atacante consegue contornar o controle?).
- Principais tipos e técnicas: SAST (análise estática do código), DAST (análise dinâmica da aplicação em execução), SCA (análise de dependências), varredura de segredos, testes de invasão (*pentest*), *fuzzing*, revisão de código e testes automatizados de casos de abuso.

- Abordagens caixa-branca, caixa-cinza e caixa-preta, e em que momento do ciclo de vida cada uma se aplica.
- Referenciais: OWASP Top 10, OWASP API Security Top 10, OWASP Web Security Testing Guide (WSTG) e OWASP Top 10 para LLM.

2.4 Qual o propósito?

- Identificar vulnerabilidades cedo, quando a correção é mais barata (*shift-left*).
- Garantir os pilares confidencialidade, integridade e disponibilidade dos dados e serviços.
- Verificar requisitos não funcionais de segurança e atender legislação de proteção de dados (LGPD), especialmente relevante em sistemas que armazenam dados de estudantes.
- Evitar regressões de segurança: transformar cada falha encontrada em teste automatizado que roda a cada alteração do código.
- Gerar evidências objetivas (relatórios) para a equipe, o cliente e auditorias.

2.5 Principais vantagens e desvantagens

O tópico será apresentado em formato de quadro comparativo, complementado pela experiência da equipe durante a implementação (por exemplo, quantidade de falsos positivos observados em cada ferramenta).

Vantagens (prós)	**Desvantagens (contras)**
Redução do custo de correção ao detectar falhas antes da produção.	Nenhuma ferramenta cobre tudo; falhas de lógica de negócio exigem testes manuais ou específicos.
Automação no CI/CD: verificação contínua a cada commit e prevenção de regressões.	Falsos positivos (principalmente em SAST) consomem tempo de triagem.
Proteção de dados de usuários, reputação e conformidade com a LGPD.	DAST e testes de invasão exigem ambiente em execução e podem ser lentos.
Ferramentas maduras e gratuitas (OWASP ZAP, Bandit, Semgrep, pip-audit, Gitleaks).	Exigem conhecimento especializado para configurar e interpretar resultados.
Gera evidências objetivas e aumenta a cultura de segurança no time.	Testes ativos em ambiente errado podem causar danos ou violar termos de uso; requerem autorização.

2.6 Exemplos de ferramentas e frameworks

Serão apresentadas as ferramentas por categoria, destacando as que serão de fato usadas no exemplo prático (marcadas com asterisco):

Categoria	**Ferramentas**
SAST (código-fonte)	Bandit*, Semgrep*, SonarQube, CodeQL

Categoria	Ferramentas
DAST (aplicação em execução)	OWASP ZAP*, Burp Suite, Nikto
SCA (dependências)	pip-audit*, OWASP Dependency-Check, Snyk, Dependabot
Segredos no repositório	Gitleaks*, TruffleHog
Testes automatizados de segurança	pytest* + httpx/TestClient do FastAPI*, PyJWT* (geração de tokens de ataque)
Orquestração (CI/CD)	GitHub Actions*
Referenciais / checklists	OWASP Top 10, OWASP API Security Top 10, OWASP WSTG, OWASP ASVS, OWASP Top 10 para LLM

2.7 Ilustração do exemplo prático

Será apresentado um diagrama da arquitetura da NotasAPI indicando a superfície de ataque, os pontos onde cada técnica de teste atua e o fluxo da esteira de CI. Um primeiro esboço dessa ilustração encontra-se na Figura 1 (Seção 3.3) e será refinado para a apresentação, acompanhado de um fluxograma da pipeline (commit → SAST/SCA/segredos → testes pytest → DAST → relatório).

2.8 Demonstração do exemplo prático

A demonstração seguirá o roteiro abaixo, com duração estimada de 8 a 10 minutos:

- **Passo 1:** subir o ambiente de teste local (docker-compose) e mostrar a API documentada em /docs.
- **Passo 2:** executar a pipeline no GitHub Actions e explicar cada etapa (SAST, SCA, segredos, testes pytest, DAST com ZAP).
- **Passo 3:** executar a pipeline na branch *vulneravel* e mostrar as ferramentas e os testes falhando, com os relatórios gerados.
- **Passo 4:** explorar ao vivo duas falhas na versão vulnerável: um aluno lendo a nota de outro aluno (IDOR) e um aluno se promovendo a professor via *mass assignment*.
- **Passo 5:** executar a pipeline na branch *main* (corrigida), mostrar a pipeline aprovada e comparar o "antes e depois" de cada correção.

2.9 Referências

Conforme orientação da disciplina, serão utilizados somente livros, artigos acadêmicos/científicos e sites oficiais das ferramentas, frameworks e APIs adotados. A lista inicial encontra-se ao final deste documento e será ampliada durante a pesquisa.

3 CONTEXTO DESCRITIVO DO PROBLEMA E DA SOLUÇÃO

3.1 Cenário: a NotasAPI

Para o exemplo prático, considera-se o cenário de uma escola técnica fictícia que precisa de uma API para que professores lancem notas e alunos consultem o próprio desempenho. A NotasAPI atende a essa necessidade com dois papéis de usuário, **ALUNO** e **PROFESSOR**, autenticação por token JWT emitido pela própria API e persistência em banco SQLite acessado via SQLAlchemy. Todo o ambiente sobe localmente com docker-compose. O Quadro 1 apresenta as rotas previstas.

Quadro 1 - Rotas da NotasAPI

Método e rota	Descrição	Quem pode acessar
POST /auth/registro	Cadastra um usuário (sempre com papel ALUNO)	Público
POST /auth/login	Valida e-mail e senha e devolve um token JWT com id, papel e expiração	Público
GET /notas/minhas	Lista as notas do aluno autenticado	ALUNO
GET /notas/{id}	Detalha uma nota	Dono da nota ou PROFESSOR
POST /notas	Lança uma nota para um aluno	PROFESSOR
GET /alunos?nome=	Busca alunos pelo nome	PROFESSOR
PATCH /usuarios/me	Atualiza nome e e-mail do próprio usuário	Usuário autenticado

Fonte: elaborado pelos autores (2026).

3.2 O problema

Notas e dados de alunos são informações pessoais protegidas pela LGPD, e uma falha que permita a um aluno ler ou alterar notas compromete a confiabilidade de todo o sistema. Na prática, APIs como essa costumam ser construídas com foco nos requisitos funcionais, com testes que verificam apenas se as rotas "funcionam". Para reproduzir esse cenário, a primeira versão da NotasAPI (branch *vulneravel*) será escrita com falhas comuns, inseridas de propósito:

- **Autenticação fraca:** segredo do JWT fixo no código e com valor fraco; token aceito sem a devida validação da assinatura e da expiração.
- **Falha de autorização a objeto (IDOR/BOLA):** GET /notas/{id} não verifica se a nota pertence ao aluno autenticado, bastando trocar o número do id para ler a nota de outro aluno.

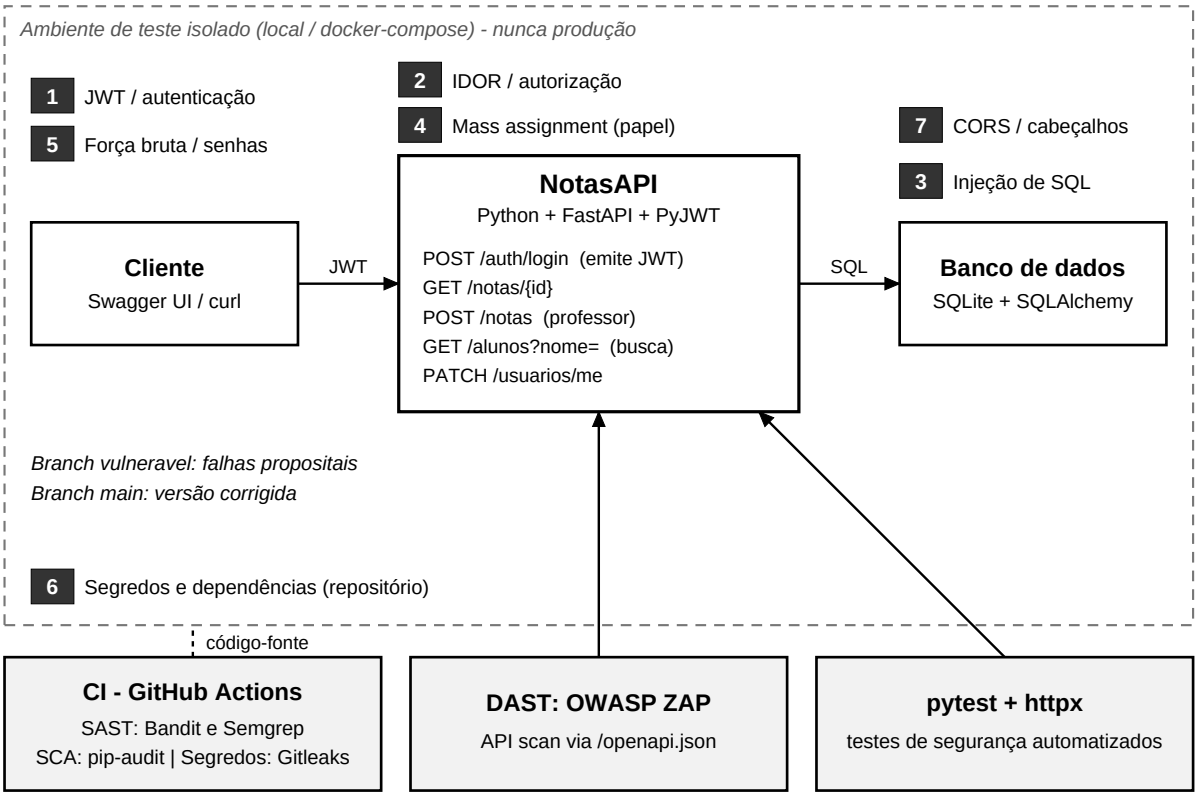
- **Falha de autorização por função:** POST /notas não verifica o papel, permitindo que um aluno lance notas.
- **Mass assignment:** PATCH /usuarios/me aceita qualquer campo do corpo da requisição, inclusive "papel", permitindo que o aluno se promova a professor.
- **Injeção de SQL:** a busca GET /alunos?nome= monta a consulta concatenando o texto recebido.
- **Senhas e login:** senhas armazenadas com MD5 sem salt e login sem limite de tentativas, permitindo ataques de força bruta.
- **Configuração e dependências:** CORS liberado para qualquer origem, ausência de cabeçalhos de segurança e uma dependência em versão com vulnerabilidade conhecida (CVE).

A pergunta que o exemplo prático responde é: **quais dessas falhas cada técnica de teste de segurança consegue detectar automaticamente, antes de o código chegar à produção?**

3.3 A solução proposta

A solução consiste em aplicar uma estratégia de testes de segurança em camadas, executada automaticamente a cada commit em uma esteira de integração contínua no GitHub Actions. O repositório terá duas branches: *vulneravel*, com as falhas descritas na Seção 3.2, e *main*, com as correções (verificação de assinatura e expiração do JWT com segredo lido de variável de ambiente, checagem de dono e de papel, esquema de entrada restrito, consultas parametrizadas, senhas com bcrypt, limite de tentativas de login, CORS restrito e dependências atualizadas). Os testes rodam apenas no ambiente local isolado. A Figura 1 ilustra a arquitetura, a superfície de ataque e onde cada técnica atua.

Figura 1 - Arquitetura da NotasAPI, pontos de teste e ferramentas



Fonte: elaborado pelos autores (2026).

As camadas da solução são:

- **Análise estática (SAST):** Bandit e Semgrep sobre o código Python, buscando padrões inseguros (segredo fixo no código, SQL montado por concatenação, uso de MD5).
- **Análise de dependências (SCA):** pip-audit sobre o requirements.txt.
- **Varredura de segredos:** Gitleaks no histórico do repositório.
- **Testes automatizados de segurança:** casos de abuso escritos com pytest e o cliente de testes do FastAPI, gerando tokens maliciosos com PyJWT. Na branch *vulneravel* esses testes falham; na *main*, passam, e passam a funcionar como testes de regressão.
- **Análise dinâmica (DAST):** OWASP ZAP em modo API scan, importando a especificação OpenAPI gerada pelo FastAPI e executando contra a API em execução no ambiente de teste.

3.4 Casos de teste planejados

O Quadro 2 resume os casos de teste que compõem o exemplo prático, associados às categorias do OWASP. A lista poderá ser ajustada durante a implementação.

Quadro 2 - Casos de teste de segurança planejados

ID	Cenário de ataque	Resultado esperado (branch main)	Técnica / ferramenta	Referência
CT01	Requisição a rota protegida sem token	HTTP 401	pytest	API2
CT02	Token assinado com segredo diferente	HTTP 401	pytest + PyJWT	API2
CT03	Token com algoritmo "none" ou papel alterado no payload	HTTP 401	pytest + PyJWT	API2
CT04	Token expirado	HTTP 401	pytest + PyJWT	API2
CT05	Aluno tentando lançar nota (POST /notas)	HTTP 403	pytest	API5
CT06	Aluno A lendo nota do aluno B (GET /notas/{id})	HTTP 403 ou 404, sem vazamento	pytest	API1
CT07	Envio de "papel": "PROFESSOR" em PATCH /usuarios/me	Campo ignorado ou HTTP 422; papel inalterado	pytest	API3
CT08	Injeção de SQL em GET /alunos?nome=	Nenhum dado extra retornado	pytest + ZAP + Bandit	A03
CT09	Várias tentativas de login com senha errada	HTTP 429 após o limite	pytest	API4 / A07
CT10	Senha gravada no banco após o cadastro	Hash bcrypt, nunca texto puro ou MD5	pytest + Bandit	A02
CT11	Requisição de origem não autorizada (CORS)	Sem cabeçalho Allow-Origin	pytest + ZAP	A05
CT12	Segredos versionados no repositório	Nenhum segredo encontrado	Gitleaks	A05
CT13	Dependências com CVE conhecida	Nenhuma CVE alta/crítica	pip-audit	A06
CT14	Padrões inseguros no código Python	Nenhum achado de severidade alta	Bandit / Semgrep	SAST
CT15	Cabeçalhos de segurança e exposição de /docs	Alertas tratados ou justificados	OWASP ZAP	A05

Legenda: API = OWASP API Security Top 10 (2023); A = OWASP Top 10 (2021). Fonte: elaborado pelos autores (2026).

3.5 Resultados esperados

- Repositório público com a NotasAPI nas duas branches, os testes de segurança, a pipeline configurada e instruções de execução.
- Relatórios gerados pelas ferramentas (ZAP, Bandit, Semgrep, pip-audit, Gitleaks) anexados à pipeline.
- Comparação "antes e depois" entre a branch *vulneravel* e a *main*.

- Análise crítica sobre o que cada técnica detectou ou deixou passar (por exemplo, falhas de lógica como IDOR tendem a escapar de SAST e DAST e só aparecem nos testes escritos pela equipe), alimentando o tópico de vantagens e desvantagens da apresentação.

4 CRONOGRAMA ATÉ A APRESENTAÇÃO

Período	Atividade
22/10/2026	Entrega deste roteiro.
23/10 a 05/11	Pesquisa teórica (introdução, histórico, conceitos, propósito, prós e contras); preparação do repositório e do ambiente de teste local.
06/11 a 15/11	Implementação da NotasAPI (branches vulneravel e main), dos casos de teste CT01 a CT15 e da pipeline no GitHub Actions.
16/11 a 22/11	Execução, coleta de relatórios, ilustração final e montagem dos slides.
23/11 a 25/11	Revisão das referências e ensaio da apresentação e da demonstração.
26/11/2026	Apresentação em horário de aula.

REFERÊNCIAS

ANDERSON, J. P. **Computer security technology planning study**. Bedford: U.S. Air Force Electronic Systems Division, 1972. (ESD-TR-73-51).

BCRYPT. **bcrypt**. Python Cryptographic Authority, [s.d.]. Disponível em: https://pypi.org/project/bcrypt/. Acesso em: 22 out. 2026.

BANDIT. **Bandit documentation**. PyCQA, [s.d.]. Disponível em: https://bandit.readthedocs.io/. Acesso em: 22 out. 2026.

FASTAPI. **Testing**. [s.d.]. Disponível em: https://fastapi.tiangolo.com/tutorial/testing/. Acesso em: 22 out. 2026.

FELDERER, M. *et al.* Security testing: a survey. **Advances in Computers**, v. 101, p. 1-51, 2016.

GITHUB. **GitHub Actions documentation**. [s.d.]. Disponível em: https://docs.github.com/actions. Acesso em: 22 out. 2026.

GITLEAKS. **Gitleaks**. [s.d.]. Disponível em: https://github.com/gitleaks/gitleaks. Acesso em: 22 out. 2026.

HOWARD, M.; LIPNER, S. **The security development lifecycle**. Redmond: Microsoft Press, 2006.

INTERNATIONAL ORGANIZATION FOR STANDARDIZATION. **ISO/IEC 25010**: systems and software engineering - systems and software quality requirements and evaluation (SQuaRE) - product quality model. Geneva: ISO, 2023.

MCGRAW, G. **Software security**: building security in. Boston: Addison-Wesley, 2006.

MYERS, G. J.; SANDLER, C.; BADGETT, T. **The art of software testing**. 3. ed. Hoboken: John Wiley & Sons, 2011.

OWASP FOUNDATION. **OWASP API Security Top 10**. 2023. Disponível em: https://owasp.org/API-Security/. Acesso em: 22 out. 2026.

OWASP FOUNDATION. **OWASP Top 10 for Large Language Model Applications**. 2025. Disponível em: https://genai.owasp.org/. Acesso em: 22 out. 2026.

OWASP FOUNDATION. **OWASP Top Ten**. 2021. Disponível em: https://owasp.org/www-project-top-ten/. Acesso em: 22 out. 2026.

OWASP FOUNDATION. **OWASP Web Security Testing Guide**. Versão 4.2. 2020. Disponível em: https://owasp.org/www-project-web-security-testing-guide/. Acesso em: 22 out. 2026.

PIP-AUDIT. **pip-audit**. Python Packaging Authority, [s.d.]. Disponível em: https://github.com/pypa/pip-audit. Acesso em: 22 out. 2026.

POTTER, B.; MCGRAW, G. Software security testing. **IEEE Security & Privacy**, v. 2, n. 5, p. 81-85, 2004.

PYJWT. **PyJWT documentation**. [s.d.]. Disponível em: https://pyjwt.readthedocs.io/. Acesso em: 22 out. 2026.

PYTEST. **pytest documentation**. [s.d.]. Disponível em: https://docs.pytest.org/. Acesso em: 22 out. 2026.

SCARFONE, K. *et al.* **Technical guide to information security testing and assessment**. Gaithersburg: National Institute of Standards and Technology, 2008. (NIST Special Publication 800-115).

SEMGREP. **Semgrep documentation**. [s.d.]. Disponível em: https://semgrep.dev/docs/. Acesso em: 22 out. 2026.

SPAFFORD, E. H. The internet worm program: an analysis. **ACM SIGCOMM Computer Communication Review**, v. 19, n. 1, p. 17-57, 1989.

STUTTARD, D.; PINTO, M. **The web application hacker's handbook**: finding and exploiting security flaws. 2. ed. Indianapolis: Wiley, 2011.

ZAP. **ZAP documentation**. [s.d.]. Disponível em: https://www.zaproxy.org/docs/. Acesso em: 22 out. 2026.